Method Return Values

Every method you call returns a value (although there's no rule that says you have to use that value). The value of a method is the value of the last expression executed by the method:

```
def method_one
  "one"
end

method_one     # => "one"

def method_two(arg)
  case
  when arg > 0  then "positive"
  when arg < 0  then "negative"
  else               "zero"
  end
end
method_two(23) # => "positive"
method_two(0)  # => "zero"
```

Ruby has a return statement, which exits from the currently executing method. The value of a return is the value of its argument(s). It is idiomatic Ruby to omit the return if it isn't needed, as shown by the previous two examples.

This next example uses return to exit from a loop inside the method:

```
def method_three
  100.times do |num|
    square = num*num
    return num, square if square > 1000
  end
end
method_three # => [32, 1024]
```

As the last case illustrates, if you give return multiple parameters, the method returns them in an array. You can use parallel assignment to collect this return value:

```
num, square = method_three
num       # => 32
square    # => 1024
```

Splat! Expanding Collections in Method Calls

We've seen that if you prefix the name of a method argument with an asterisk, multiple arguments in the call to the method will be passed as an array. Well, the same thing works in reverse.

When you call a method, you can convert any collection or enumerable object, or object that implements to_a into its constituent elements and pass those elements as individual parameters to the method. Do this by prefixing array arguments with an asterisk:

```
def five(a, b, c, d, e)
  "I was passed #{a} #{b} #{c} #{d} #{e}"
end

five(1, 2, 3, 4, 5 )       # => "I was passed 1 2 3 4 5"
five(1, 2, 3, *['a', 'b']) # => "I was passed 1 2 3 a b"
```